Flamme — Build Plan

From prototype to evidence platform: stack, architecture and step-by-step build

Team Flamme · NASA Space Apps · Flame in Freefall

October 2026

Contents

0. What we are building 1

1. The stack 2

2. Repository layout (target) 3

3. The core data model (do this first — everything depends on it) 3
   Database tables . . . 4

4. Step-by-step build 4
   Step 1 — Clean up the repo and set up tooling (½ week) . . . 5
   Step 2 — Define the schema in code (½ week) . . . 5
   Step 3 — Table ingestion pipeline (1½ weeks) → Deliverable 1 . . . 5
   Step 4 — Report (PDF) extraction with human review (1½ weeks) → Deliverable 1 . . . 5
   Step 5 — Search and compare (1 week) → Deliverable 2 . . . 6
   Step 6 — Gap map (1 week) → Deliverable 3 . . . 6
   Step 7 — Fit models to the data (2 weeks) → Deliverable 4 (science part) . . . 6
   Step 8 — Ground the 3D lab in data (1½ weeks) → Deliverable 4 (visual part) . . . 7
   Step 9 — Reporting template (½ week) → Deliverable 5 . . . 7
   Step 10 — Outreach mode (½ week) → Deliverable 6 . . . 7
   Step 11 — Quality, accessibility, deploy (1 week, overlapping) . . . 8

5. Timeline summary 8

6. Suggested team split 8

7. Risks and how to handle them 8

0. What we are building

The plan document defines six deliverables on top of the existing prototype:

#	Deliverable (from the plan)	What it means technically
1	Evidence database	Every NASA experiment stored in the ten-condition schema, with source link and per-field “not reported” flags
2	Search and compare	Filter by any condition (“PMMA, < 21 % O_2, opposed flow < 10 cm/s”), side-by-side view in standard units
3	Gap map	Coverage plot of tested vs untested condition combinations
4	Data-grounded 3D lab	Replace hand-tuned curves in `labModel.js` with relationships fitted to the database; show the nearest real experiments next to the simulation

#	Deliverable (from the plan)	What it means technically
5	Reporting template	The ten-condition format published as a JSON Schema + fillable template
6	Outreach mode	Keep the 92-second guided Simulate sweep

What already exists in the repo and is reused (not rewritten):

- `src/Lab.jsx`, `src/labScene.js`, `src/labModel.js` — Three.js flame lab and 92-s timeline.
- `src/data.js`, `src/science.js` — the field model (`source / unit / normalized / status`) and transparent matching. This field model becomes the database schema.
- `flame_in_freefall_data/psi_scraper.py`, `download_ntrs.py` — PSI and NTRS harvesting (24 investigations, ~3,000 table rows, 36 PDFs).
- `findings/` — table → finding extraction and the PDF extraction utility.
- `parameter_catalog/` — the ten-condition reference.

Guiding rule for every step: a value is either Observed (cited source row/page), Predicted (named model version + uncertainty), or Unknown. Nothing is ever silently filled in.

1. The stack

Layer	Choice	Why
Frontend	React 19 + Vite 7 + TypeScript (migrate the current JS gradually)	Already used; TS catches schema mismatches between API and UI
3D	Three.js (existing `labScene.js`)	Already built; no need for react-three-fiber rewrite
Routing / data fetching	TanStack Router + TanStack Query	Typed URLs (shareable search filters), caching
UI state (lab sliders)	Zustand	Small, works outside React (Three.js loop can read it)
Tables	TanStack Table	Sortable/filterable compare view
Charts (gap map, fits)	Observable Plot (or Apache ECharts)	Heatmaps and scatter with little code
Styling	Existing `app.css` → CSS variables; Lucide icons	Keep current look
API	Python 3.12 + FastAPI + Pydantic v2	Pipeline and ML are already Python; Pydantic models generate the JSON Schema for deliverable 5 for free
ORM / migrations	SQLAlchemy 2 + Alembic	Versioned schema changes
Database	PostgreSQL 16 (hosted on Neon or Supabase, free tier)	Relational data, JSONB for raw source rows, range queries
Units	pint	Convert atm/psi/kPa, %/mole fraction, mm/cm, °C/K safely
Data wrangling	pandas (or polars), pandera for validation	Per-investigation adapters + schema checks
PDF extraction	PyMuPDF + pdfplumber (tables) + an LLM with structured (JSON-schema) output	Pull conditions from NTRS reports with page + quote; human review required
ML	scikit-learn, statsmodels, optional PyMC for Bayesian fits	Small data → simple, interpretable, calibrated models
Experiment tracking	MLflow (local) or plain versioned JSON in `models/`	Every fit has a version the UI can cite
Data versioning	DVC (remote: Google Drive or S3/R2)	239 MB of raw data must not live in git
Auth (reviewers only)	Supabase Auth (or Clerk)	Public site is read-only; only reviewers log in

Layer	Choice	Why
Tests	Vitest (JS), Playwright (browser), pytest (API/pipeline/ML)	Existing `node --test` moves to Vitest
CI	GitHub Actions	Lint, test, build, data validation on every PR
Hosting	Frontend on Vercel; API on Render / Fly.io / Railway; DB on Neon	Free tiers, push-to-deploy

Cheaper alternative for the hackathon window: skip the live API. Run the pipeline at build time, export `experiments.json` + `models/*.json`, and query in the browser with DuckDB-WASM. Same schema, same pipeline — just no server. Move to FastAPI + Postgres when you need the reviewer workflow (step 6).

2. Repository layout (target)

```
flamme/
├── apps/
│   ├── web/                  React + Vite + TS (moved from src/)
│   │   ├── src/lab/          Lab.tsx, labScene.ts, labModel.ts
│   │   ├── src/search/       filter bar, results, compare view
│   │   ├── src/gaps/         gap map
│   │   └── src/api/          generated types + TanStack Query hooks
│   └── api/                  FastAPI
│       ├── flamme_api/
│       │   ├── models.py     SQLAlchemy tables
│       │   ├── schemas.py    Pydantic (source of truth for the 10 conditions)
│       │   ├── routes/       experiments, search, gaps, models, review
│       │   └── query.py      filter → SQL
│       └── alembic/
├── pipeline/
│   ├── harvest/              psi_scraper.py, download_ntrs.py (moved)
│   ├── adapters/            one file per investigation: saffire2.py, bass2.py, flex.py …
│   ├── extract_pdf/          NTRS report extraction (LLM + quote + page)
│   ├── normalize.py          units via pint, vocabularies, status flags
│   ├── validate.py           pandera checks
│   └── load.py               upsert into Postgres
├── ml/
│   ├── features.py
│   ├── fit_flammability.py
│   ├── fit_spread.py
│   ├── fit_suppression.py
│   └── export.py             → models/<name>@<version>.json for the lab
├── schema/
│   └── flamme-experiment.schema.json   (generated, deliverable 5)
├── data/                     DVC-tracked raw + processed
└── docs/
```

3. The core data model (do this first — everything depends on it)

Every condition value is a Field — exactly the shape already used in `src/data.js`:

```python
class Field(BaseModel):
    source: str | None          # verbatim from the source, e.g. "~ 22.1", "037 mm", "Insignifica
```

```
    value: float | str | None   # normalized value, only when defensible
    unit: str | None            # standard unit (see table below)
    status: Literal["reported", "approximate", "qualitative",
                    "ambiguous", "not_reported", "not_applicable", "derived"]
    note: str | None = None
```

The ten conditions, each with one standard unit:

#	Condition	Fields (standard unit)
1	Material / fuel	name, material_class (enum: cellulosic, PMMA, silicone, fabric-composite, liquid droplet, gas…), formulation, coating, substrate
2	Geometry	length, width, thickness (mm); thermally thin/thick flag (derived)
3	Gravity	regime (earth / lunar / martian / microgravity), g level (g_0), platform (ISS, Cygnus, drop tower, parabolic)
4	Oxygen	X_{O_2} (% vol)
5	Pressure	P (kPa)
6	Forced airflow	u (cm/s)
7	Flow direction	concurrent / opposed / cross / quiescent
8	Ignition	igniter type, power (W), duration (s), position
9	Initial temperature	T_0 (K)
10	Suppression	agent, concentration (% vol), flow rate, timing (s after ignition)

Derived column: oxygen partial pressure, because the plan says this, not percentage, drives burning:

$$p_{O_2} = \frac{X_{O_2}}{100} P$$

It is stored with `status = "derived"` and only when both inputs are reported.

Database tables

Table	Key columns
`investigation`	psi_id, name, platform, doi, url
`source_document`	id, investigation_id, kind (table/report/image), filename, url, sha256, version
`experiment`	id, investigation_id, source_document_id, source_row / page, quote, ten condition columns (each `value`, `unit`, `status`, `source`), raw_row JSONB
`outcome`	experiment_id, metric (enum: spread_rate, burn_time, extinction, ignited, flame_length, …), value, unit, status, source
`review`	experiment_id, reviewer, decision (approved / corrected / rejected), diff JSONB, timestamp
`model_version`	name, version, trained_on (dataset hash), metrics JSONB, domain JSONB, created_at
`audit_log`	every normalization decision and correction (this is the “agentic memory” from the README, done as an auditable table first)

Store condition columns flat (e.g. `oxygen_value`, `oxygen_status`, `oxygen_source`) rather than one JSON blob — flat columns are what make range search and the gap map fast and simple.

4. Step-by-step build

Each step ends with a Done when check. Rough sizing assumes 2–4 people.

Step 1 — Clean up the repo and set up tooling (½ week)

1. Create the `apps/web`, `apps/api`, `pipeline`, `ml`, `schema` folders; move files with `git mv` so history is kept.
2. Web: `npm create vite@latest` settings merged into the existing project; add TypeScript (`allowJs: true` so old `.js` keeps working), Vitest, ESLint, Prettier.
3. API: `uv init apps/api`, add `fastapi uvicorn sqlalchemy alembic psycopg[binary] pydantic pint pandas pandera`.
4. `dvc init`; `dvc add data/raw` (move `flame_in_freefall_data/psi_data` and `ntrs_pdfs` there); configure a remote.
5. Add `LICENSE` (the README notes there is none) and a `.env.example`.
6. GitHub Actions: `web.yml` (lint, vitest, build), `api.yml` (ruff, pytest).

Done when: `npm run dev`, `npm test`, `uv run pytest` and CI all pass on a clean clone; existing 5 science tests still green.

Step 2 — Define the schema in code (½ week)

1. Write `schemas.py` (Pydantic) for `Field`, the ten conditions, `Outcome`, `Experiment`.
2. Write SQLAlchemy models + first Alembic migration.
3. Generate `schema/flamme-experiment.schema.json` with `Experiment.model_json_schema()`.
4. Generate TypeScript types for the web app from the FastAPI OpenAPI spec: `npx openapi-typescript http://localhost:8000/openapi.json -o apps/web/src/api/types.ts`.
5. Port the five curated records in `src/data.js` into a seed file and load them — they are the golden test set.

Done when: the 5 seed records round-trip DB → API → UI unchanged, including `037 mm` staying `ambiguous` and pressure staying `not_reported`.

Step 3 — Table ingestion pipeline (1½ weeks) → Deliverable 1

The PSI experimental tables all have different columns, so write one adapter per investigation, not one generic parser.

1. For each of the 22 table files, write `pipeline/adapters/<investigation>.py` with a `COLUMN_MAP` (source column → condition) and a `parse_row()` that returns `Experiment` objects with `source` kept verbatim.
2. `normalize.py`: convert with pint; map vocabularies (e.g. “Concurrent”, “co-current”, “cocurrent” → `concurrent`); set `status` (~ → approximate, words → qualitative, unparseable → ambiguous, blank → not_reported).
3. `validate.py` with pandera: O_2 in 0–100, P in 0–200 kPa, u ≥ 0, thickness > 0, every row has a source file + row number.
4. `load.py`: idempotent upsert keyed on (source_document, source_row).
5. Reuse what `findings/build_findings.py` already parses — lift that logic into the adapters.

Prioritise investigations the plan names: Saffire (PSI-99 etc.), BASS / BASS-II (PSI-25), FLEX (PSI-69), then the rest.

Done when: `make ingest` loads all table investigations; a pytest per adapter checks row counts and 3 hand-verified rows; validation report shows per-field “not reported” rates.

Step 4 — Report (PDF) extraction with human review (1½ weeks) → Deliverable 1

Many conditions (pressure, ignition energy, flow direction) only appear in NTRS reports and drop-tower / parabolic-flight papers, and NASA-STD-6001 test reports.

1. `extract_pdf/`: PyMuPDF for text, pdfplumber for tables; chunk by page.
2. Ask an LLM for structured output matching the Pydantic `Experiment` schema, and require a `page` and verbatim `quote` for every non-null field. Reject any field whose quote is not found in the page text (string-match check in code).
3. Everything extracted lands with `review.decision = pending`; nothing pending is shown publicly.

4. Build a small review screen (`/review`, reviewer login via Supabase Auth): PDF page on the left, extracted fields on the right, approve / correct / reject. Corrections go to `audit_log`.

Done when: at least the 36 downloaded NTRS PDFs are processed; ≥ 1 reviewer has approved records from each; a test confirms unapproved rows never appear in public API responses.

Step 5 — Search and compare (1 week) → Deliverable 2

1. API: `GET /experiments?material_class=PMMA&o2_lt=21&direction=opposed&flow_lt=10&gravity=microgr` Map each filter to SQL on the flat columns.
2. Decide and show how unknowns behave: by default a row with `oxygen_status = not_reported` is excluded from an O_2 filter but counted in a "12 more records have unknown O_2" link.
3. Free-text bar: a deterministic parser (regex + vocabulary) turns "PMMA, under 21% O2, opposed flow below 10 cm/s" into the filter object and shows it back as editable chips. An optional LLM fallback may propose filters, but the user always sees the chips before searching.
4. Keep `science.js` similarity ranking as "closest experiments", now run server-side over the whole DB.
5. Compare view (TanStack Table): up to 4 experiments as columns, the ten conditions as rows, standard units, unknown cells visibly marked, each cell links to the source row/page.
6. Filters live in the URL so searches can be shared.

Done when: the plan's example query returns the right rows; Playwright test covers search → compare → open source link.

Step 6 — Gap map (1 week) → Deliverable 3

1. API `GET /gaps?x=oxygen&y=flow&facet=gravity&material_class=…` returns counts per bin. Default bins: O_2 in 2 %-points, flow in 5 cm/s, pressure in 10 kPa.
2. Frontend heatmap (Observable Plot): coloured cells = tested (count shown), empty outlined cells = gap, hatched = only records with approximate values.
3. Click a cell → search results for that bin. Facet selector: gravity, direction, material class.
4. Overlay the fitted flammability boundary from Step 7 so users see "untested and near the boundary" — the most useful place for a next experiment.

Done when: the map reproduces counts from a direct SQL query (pytest), and clicking any tested cell returns exactly that many rows.

Step 7 — Fit models to the data (2 weeks) → Deliverable 4 (science part)

Data is small and heterogeneous, so use simple, physics-informed, interpretable models — one per question, never one model for everything.

Model	Inputs	Target	Method
Flammability boundary	p_{O_2}, u, material class, gravity	ignited & sustained (yes/no)	Logistic regression or Gaussian-process classifier
Spread rate	thickness, p_{O_2}, u, direction	spread rate (mm/s), only where the source measures the same quantity	Physics-form regression (below)
Suppression	agent, concentration, u	extinguished (yes/no), time to extinction	Logistic regression / survival model

Physics form for spread over a thermally thin sample in opposed flow (de Ris), used as the model shape instead of a black box:

$$V_f = \frac{\pi}{4}\,\frac{\lambda_g\,(T_f - T_v)}{\rho_s\,c_s\,\tau\,(T_v - T_\infty)}$$

so spread rate should scale as $V_f \propto \frac{1}{\tau}$ with thickness τ. Fit the coefficient (and any extra oxygen/flow dependence) from data, and report how well the observed data follows this shape.

Rules for every fit:

1. Grouped cross-validation by investigation (`GroupKFold`) so the same experiment never appears in train and test.
2. Start with a baseline (mean / single-variable) and only keep a model if it beats it.
3. Report calibrated uncertainty (prediction intervals or calibrated probabilities).
4. Record the domain: the convex hull / ranges of training inputs. Outside it the UI says "outside tested conditions" instead of predicting.
5. `ml/export.py` writes `models/flammability@1.0.json` (coefficients, domain, metrics, dataset hash) and registers it in `model_version`.

Done when: each model card shows CV metrics vs baseline, the domain, and the dataset version; pytest checks export → reload gives identical predictions.

Step 8 — Ground the 3D lab in data (1½ weeks) → Deliverable 4 (visual part)

1. In `labModel.js`, the `derive(p)` function currently maps sliders to flame visuals with hand-designed curves. Split it into two layers:
   - `predict(p)` → physical quantities from the exported model JSON (ignites? probability, spread rate ± interval, extinguished?).
   - `toVisual(prediction)` → flame size, soot, shell shape, colour (keep the existing Three.js look).
2. Load models with `fetch('/models/flammability@1.0.json')`; run inference in the browser (logistic/linear models are a few lines of JS).
3. Out-of-domain → keep the illustrative curve but show a banner "Illustrative — no NASA data for these conditions".
4. Add a "Closest real experiments" side panel: call the similarity endpoint with the current slider values; show top 3 with their real outcome and source link. When a flame goes out in the lab, the panel shows whether a real experiment near those conditions also went out.
5. Every number on screen is labelled Observed / Predicted (model@version) / Unknown.

Done when: for each seed experiment, setting the sliders to its conditions shows that experiment as #1 closest, and the predicted outcome direction agrees with the observed one (or the mismatch is shown, not hidden).

Step 9 — Reporting template (½ week) → Deliverable 5

1. Publish `schema/flamme-experiment.schema.json` (already generated in Step 2) at `/schema`.
2. Generate a fillable CSV template and a web form from the same schema (`react-jsonschema-form`) that validates input and flags missing conditions in red.
3. "Validate my file" page: upload a CSV/JSON, get a completeness report (which of the ten conditions are missing).

Done when: a file exported from the database validates against the schema; a file with missing pressure gets a clear warning.

Step 10 — Outreach mode (½ week) → Deliverable 6

1. Keep the 92-second `/#studio-play` sweep; add captions that now cite real experiments ("Saffire II sample 2-5 at ~22 % O_2 …").
2. A "classroom" route with large text, no admin UI, works offline as a PWA (`vite-plugin-pwa`).

Done when: sweep runs end-to-end on a school laptop in Chrome/Firefox at ≥ 30 fps (check with the Performance panel).

Step 11 — Quality, accessibility, deploy (1 week, overlapping)

1. Tests: Vitest for `labModel/science`; Playwright for search → compare → lab → source; pytest for adapters, API, ML export; a visual snapshot of the lab.
2. Accessibility: keyboard shortcuts already exist — add `aria-live` for lab status, check with axe.
3. Performance: lazy-load the Three.js lab chunk; cap pixel ratio at 2.
4. Deploy: web → Vercel (env var `VITE_API_URL`); API → Render/Fly with `DATABASE_URL`; DB → Neon; run `alembic upgrade head` + `make ingest` in a release job.
5. Footer on every page: "Not a NASA product. Not for certification or operational safety decisions."

5. Timeline summary

Week	Steps	Milestone
1	1, 2	Tooling, schema, 5 seed records through the full stack
2–3	3	All PSI tables in the database
3–4	4, 5	Reports extracted + reviewed; search and compare live
5	6, 9	Gap map; reporting template
6–7	7	Fitted models with model cards
7–8	8, 10, 11	Data-grounded lab, outreach mode, deploy

For a hackathon-length sprint (≈ 48 h), do the static path: Steps 1–3 for Saffire, BASS and FLEX only, Step 5 + 6 with DuckDB-WASM over exported JSON, one flammability-boundary model for Step 7–8, and the closest-experiments panel. That alone demonstrates the full idea end-to-end with real data.

6. Suggested team split

Person	Owns
Data	Steps 3, 4 (adapters, normalization, PDF extraction, review)
Backend	Steps 2, 5 API, 6 API, deploy
Frontend / 3D	Steps 5 UI, 6 UI, 8, 10
ML / science	Step 7, model cards, checking physics and limitations

7. Risks and how to handle them

Risk	Mitigation
Too few comparable outcome measurements for a model	Fit per question and per material class; if data is too thin, show only nearest experiments + "insufficient data" rather than a fit
LLM extraction invents values	Quote + page required, quote string-matched in code, human approval before publishing
Unit / definition mismatch (e.g. Saffire "Spread Length" column holding mm/s)	Keep `source` verbatim, mark `ambiguous`, exclude from fits until reviewed
Leakage making models look better than they are	GroupKFold by investigation
Users treat the lab as a real fire prediction	Observed / Predicted / Unknown labels, out-of-domain banner, disclaimer footer
Raw data size (~14 TB on PSI)	Tables and reports only; raw data download remains opt-in and size-capped